1. **Find 10 different model providers and their models released in last 12 months.**

Provider	Model	Release Date	Cutoff Date	Token Cost Input	Token Cost Output	Context Window	Public or Private	Best Use Case
OpenAI	GPT-5.5	Apr-26	Nov-25	$0.005 per 1K	$0.03 per 1K	1M	Public	General agentic AI, autonomous reasoning
Anthropic	Claude Opus 4.7	May-26	Jan-26	$0.005 per 1K	$0.025 per 1K	200K+	Public	Coding, complex reasoning, agentic workflows
Google	Gemini 3.1 Pro	Feb-26	Dec-25	$0.002 per 1K	$0.012 per 1K	2M	Public	Multimodal tasks, long context, multilingual
Meta	Llama 4 Maverick	Early 2026	Nov-25	Free	Free	400K	Open-Source	Self-hosted, on-premise, community projects
xAI	Grok 4.3	May-26	Mar-26	$0.00125 per 1K	$0.0025 per 1K	2M	Public	Real-time tasks, fast reasoning, agentic workflows
DeepSeek	V4-Pro	Early 2026	Dec-25	$0.000435 per 1K (promo)	$0.00087 per 1K (promo)	1M+	Open-Source	Cost-effective coding, research, self-hosting
Alibaba	Qwen 3.6 Max	May-26	Mar-26	$0.00078 per 1K	$0.0039 per 1K	260K	Public	Coding champion, cost-

								effective, multilingual
NVIDIA	Nemotron 3 Omni	May-26	Feb-26	Custom	Custom	30B params	Open-Source	Multimodal creative work, on-device inference
Mistral	Mistral Large 2	2025-2026	Varies	$0.0015-0.005 per 1K	$0.005-0.015 per 1K	256K	Mixed	Enterprise EU, code generation, research
Anthropic	Claude Sonnet 4.6	Feb-26	Jan-26	$0.003 per 1K	$0.015 per 1K	200K	Public	Budget coding, writing, content generation

2. Read 2-3 articles about how transformer works?

ANS: How Transformers work for Cybersecurity

1. INTRODUCTION TO CYBERSECURITY CHALLENGES

Modern cybersecurity faces an unprecedented challenge: attackers no longer launch direct assaults but orchestrate sophisticated multi-stage campaigns that unfold over weeks or months, making each individual event appear innocuous to traditional defenses. Legacy security systems like rule-based Intrusion Detection Systems (IDS) examine events in isolation, comparing each against a finite database of known attack signatures, rendering them blind to novel attack patterns and coordinated multi-step operations. Advanced Persistent Threats (APTs) deliberately distribute their malicious activities across time and systems to evade detection, with initial compromise occurring hours before privilege escalation, days before lateral movement, and weeks before data exfiltration. Traditional machine learning models like Random Forests or Support Vector Machines lack the architectural sophistication to capture long-range temporal dependencies in security logs, often failing to correlate events separated by hours or days despite clear causal relationships. The cybersecurity industry urgently requires a detection mechanism that can understand attack narratives—sequences of

events that form coherent attack chains—rather than treating each event independently as the current generation of security tools do. Transformer architectures, with their revolutionary self-attention mechanism, represent a paradigm shift by enabling security systems to attend simultaneously to all events in a sequence, discovering hidden relationships and reconstructing complete attack timelines that human analysts might take days to piece together.

2. VECTOR EMBEDDINGS: CONVERTING SECURITY EVENTS TO NUMERICAL REPRESENTATIONS

Vector embeddings transform categorical and numerical security event data into dense, fixed-dimensional numerical vectors where mathematical operations become meaningful and semantic similarity can be computed through distance metrics like cosine similarity. In traditional security systems, events are stored as raw log strings or one-hot encoded categorical variables that lack any inherent structure—treating "admin access to database" identically to "user access to database" despite their vastly different risk profiles—whereas embeddings allow the model to learn that semantically similar events (like "login from China" and "login from Japan") occupy nearby regions in vector space despite different source countries. The embedding process begins by identifying relevant features of each security event such as user identity, source IP address, destination system, action type, timestamp, and behavioral anomalies, then converting each feature category into a learned vector representation through an embedding lookup table where each categorical value (e.g., "user_alice", "protocol_SSH", "port_22") maps to a unique vector learned during model training. This feature embedding approach enables the model to discover implicit security patterns—for instance, embeddings learned from training data automatically capture that "3 AM login from unusual country" should be encoded differently than "9 AM login from office"—without requiring security engineers to manually specify these contextual relationships. The dimensionality of embeddings (typically 256-512 dimensions for security applications) creates a geometric space where each security event becomes a point, and events with similar behavioral signatures, attack vectors, or anomalous characteristics naturally cluster together, allowing downstream layers to recognize attack patterns by detecting clusters of similarly-embedded events. This representation is fundamental to transformer architectures because it converts unstructured security data into a format where the self-attention mechanism can compute meaningful relationships through vector operations like dot products and cosine similarities, enabling the model to answer questions like "which events are most relevant to this data exfiltration event?" by measuring embedding similarity.

3. SELF-ATTENTION MECHANISM: DISCOVERING EVENT RELATIONSHIPS

Self-attention is a mathematical operation that computes a weighted importance score for every pair of events in a sequence, allowing the model to determine which events are most relevant to understanding any given event, fundamentally solving the problem that traditional models process events sequentially and therefore lose information about distant events that may be crucial for understanding attack chains. The mechanism operates through three learned linear transformations of each event's embedding: the Query (Q) vector represents "what am I looking for in other events?", the Key (K) vector represents "what can other events offer me?", and the Value (V) vector represents "what information should I contribute to other events?", creating a framework where the model learns to match queries against keys to determine compatibility and then aggregate values from compatible events. For each event in a sequence, the model computes an attention score against every other event by calculating the dot product of the event's Query vector with all events' Key vectors—a high dot product indicates relevance—producing an n×n matrix of scores where n is the sequence length, capturing all pairwise relationships across the entire event history. These raw attention scores are then normalized using the softmax function after dividing by the square root of the key dimension, producing attention weights that sum to 1.0 for each event, representing a probability distribution over which events are important for understanding the current event. The final step aggregates information from all events by computing a weighted sum of their Value vectors, where each value is weighted by the corresponding attention weight, producing a new enriched representation for each event that incorporates context-aware information from all relevant events in the sequence regardless of their temporal distance. This mechanism is revolutionary for cybersecurity because it directly addresses the core challenge of attack detection: a data exfiltration event at day 7 can "attend" to the initial phishing email at day 0, enabling the model to reconstruct entire attack narratives as unified entities rather than disconnected events that legacy systems treat independently.

4. FEED-FORWARD NETWORKS: CLASSIFYING ATTACK TYPES

Feed-forward networks in transformers are lightweight 2-layer neural networks applied independently to each event's representation after self-attention has captured relationship information, serving to extract task-specific features and classify the attack type based on the enriched event context derived from attention mechanisms. Each feed-forward network consists of a first dense layer that expands the input representation from its original dimension (e.g., 512 dimensions) to a larger intermediate dimension (typically 4x larger, e.g., 2048 dimensions), followed by a ReLU activation function that applies the non-linear transformation ReLU(x) = max(0, x) which zeros out all negative values and introduces non-linearity essential for learning complex decision boundaries. The expansion phase creates an information bottleneck that

forces the network to learn which features are most important by requiring all intermediate representations to flow through 2048 hidden units, similar to how a funnel forces fluid to flow through a narrow opening—this compression and expansion pattern is mathematically equivalent to learning a non-linear feature extraction process optimized for distinguishing between normal events and various attack types. After the ReLU activation and expansion, a second dense layer contracts the representation from the intermediate dimension back to the original dimension (2048 back to 512), performing dimensionality reduction that aggregates the expanded features back into a manageable size for passing to subsequent transformer layers or final classification heads. These feed-forward networks learn to recognize attack-specific signatures by amplifying features associated with particular attack types (e.g., amplifying "rapid privilege escalation indicators" for detecting privilege escalation attacks or "data volume spikes" for detecting exfiltration) while suppressing irrelevant features through the learned weight matrices. This architecture is highly efficient because the feed-forward operations are applied identically to each event independently, enabling massive parallelization across events, and the learned weights become specialized attack classifiers through training on labeled attack data where the network observes thousands of examples of each attack type and learns to extract their characteristic patterns through backpropagation and gradient descent optimization.